

1. What is Java?

Answer

Java ek High-Level, Object-Oriented, Platform Independent Programming Language hai jise Sun Microsystems ne develop kiya tha (ab Oracle maintain karta hai).

Java ka motto hai:

Write Once, Run Anywhere (WORA)

Iska matlab ek baar code likho aur JVM hone par kisi bhi OS par chala sakte ho.

Example

```
class Demo {  
    public static void main(String[] args) {  
        System.out.println("Hello Java");  
    }  
}
```

2. What are the Features of Java?

Answer

Java ki important features:

- Platform Independent
- Object-Oriented
- Secure
- Robust
- Multithreaded
- Portable
- High Performance (JIT Compiler)
- Simple
- Distributed
- Dynamic

Interview Line

Java is Simple, Secure, Object-Oriented and Platform Independent.

3. Why is Java Platform Independent?

Answer

Java source code compile hone ke baad Bytecode (.class file) me convert hota hai.

Ye bytecode directly machine par nahi chalta.

Ye JVM ke andar chalta hai.

Har Operating System ka apna JVM hota hai.

Isliye same bytecode Windows, Linux aur Mac sab par chal jata hai.

Flow

Java Code



Compiler (javac)



Bytecode (.class)



JVM



Machine Code

4. What is JVM?

Answer

JVM (Java Virtual Machine) ek virtual machine hai jo Bytecode ko Machine Code me convert karti hai.

Work

- Bytecode execute karti hai
- Memory manage karti hai
- Garbage Collection karti hai

- Platform Independence provide karti hai

Example

Program.class

↓

JVM

↓

Machine Code

5. What is JRE?

Answer

JRE = Java Runtime Environment

Ye Java Program ko Run karne ke liye hota hai.

JRE ke andar

- JVM
- Libraries
- Runtime Files

Developer ke liye nahi, sirf Run karne ke liye.

6. What is JDK?

Answer

JDK = Java Development Kit

Java Program ko Develop aur Run dono karne ke liye use hota hai.

Iske andar

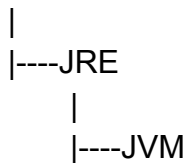
- JRE
- JVM
- Compiler (javac)
- Debugger
- Tools

7. Difference Between JVM, JRE and JDK

JDK	JRE	JVM
Develop + Run	Run only	Execute Bytecode
Compiler hai	Compiler nahi	Compiler nahi
JRE included	JVM included	Lowest Layer

Diagram

JDK



8. What is Bytecode?

Answer

Bytecode wo code hai jo Java Compiler generate karta hai.

Extension:

.class

Ye JVM execute karti hai.

Example

Hello.java

↓

javac

↓

Hello.class

9. What is Object-Oriented Programming (OOP)?

Answer

OOP ek Programming Concept hai jisme hum Real World Objects ke basis par software banate hain.

Isme Data aur Methods ek hi unit me hote hain.

Example

Car

Data:

Color

Brand

Methods:

Start()

Stop()

10. What are the Four Pillars of OOP?

Answer

1. Encapsulation
2. Inheritance
3. Polymorphism
4. Abstraction

Interview me almost fix question hai.

11. What is Class?

Answer

Class ek Blueprint ya Template hoti hai jisse Objects bante hain.

Example

```
class Student{  
  
}
```

Class me variables aur methods hote hain.

12. What is Object?

Answer

Object Class ka Instance hota hai.

Object memory occupy karta hai.

Example

```
Student s = new Student();
```

Yaha

Student → Class

s → Object

13. Difference Between Class and Object

Class

Object

Blueprint	Real Instance
-----------	---------------

Memory nahi leti	Memory leti hai
---------------------	--------------------

Design	Actual Entity
--------	---------------

Example

Class → Car

Object → BMW, Audi

14. What is Constructor?

Answer

Constructor ek Special Method hota hai jo Object create hote hi automatically call hota hai.

Rules

- Name same as Class
- No Return Type

Example

```
class Student{
```

```
    Student(){
```

```
        System.out.println("Constructor Called");
```

```
    }
```

```
}
```

Output

Constructor Called

15. Types of Constructors

1. Default Constructor

```
Student(){  
  
}
```

2. Parameterized Constructor

```
Student(String name){  
  
}
```

16. Difference Between Default and Parameterized Constructor

Default	Parameterized
No Parameters	Parameters hote hain
Default values	Custom values

Example

```
Student(){  
  
}  
  
Student(String name){  
  
}
```

17. What is **this** Keyword?

Answer

this current object ko refer karta hai.

Use

- Current object
- Current variable
- Current method
- Constructor call

Example

```
class Student{  
  
    String name;  
  
    Student(String name){  
  
        this.name = name;  
  
    }  
  
}
```

Yaha

this.name → Instance Variable

name → Local Variable

18. What is **super** Keyword?

Answer

super Parent Class ko refer karta hai.

Use

- Parent Constructor
- Parent Variable
- Parent Method

Example

```
class Animal{  
  
    void sound(){
```

```
System.out.println("Animal");
```

```
}
```

```
}
```

```
class Dog extends Animal{
```

```
void show(){
```

```
super.sound();
```

```
}
```

```
}
```

Output

Animal

19. What is **static** Keyword?

Answer

static member Class se belong karta hai, Object se nahi.

Static Variable

```
class Demo{
```

```
static int a=10;
```

```
}
```

Static Method

```
class Demo{
```

```
static void show(){
```

```
System.out.println("Hello");
```

```
}
```

```
}
```

Call

```
Demo.show();
```

Object banane ki zarurat nahi.

20. What is **final** Keyword?

Answer

final ka matlab hai change nahi ho sakta.

Final Variable

```
final int a=10;
```

```
a=20;
```

Output

Compile Time Error

Final Method

Override nahi kar sakte.

```
final void show(){
```

```
}
```

Final Class

Inherit nahi kar sakte.

```
final class Demo{
```

```
}
```

21. Difference between `final`, `finally`, and `finalize()`?

final	finally	finalize()
Keyword	Block	Method
Variable, Method, Class ke saath use hota hai	Exception handling me use hota hai	Garbage Collector call karta tha
Value change nahi hoti	Hamesha execute hota hai	Java 9+ se Deprecated

Example

```
final int a = 10;
// a = 20; // Error
try {
    int a = 10 / 0;
} catch (Exception e) {
    System.out.println("Error");
} finally {
    System.out.println("Always Execute");
}
```

22. What is Method Overloading?

Answer

Ek hi class me same method name but different parameters.

Compile Time Polymorphism.

Example

```
class Demo {

    void add(int a, int b) {
        System.out.println(a + b);
    }
}
```

```
}

void add(int a, int b, int c) {
    System.out.println(a + b + c);
}
}
```

23. What is Method Overriding?

Answer

Child class Parent class ke method ko apne hisab se dobara likhti hai.

Run Time Polymorphism.

Example

```
class Animal {
    void sound() {
        System.out.println("Animal Sound");
    }
}
```

```
class Dog extends Animal {

    @Override
    void sound() {
        System.out.println("Bark");
    }
}
```

24. Difference Between Overloading and Overriding

Overloading	Overriding
Same Class	Parent & Child
Method Name Same	Method Name Same

Parameters Different	Parameters Same
-------------------------	-----------------

Compile Time	Run Time
--------------	----------

25. What is Compile-Time Polymorphism?

Compiler compile time par decide karta hai kaunsa method chalega.

Ye Method Overloading se achieve hota hai.

Example

```
add(10,20);
```

```
add(10,20,30);
```

26. What is Run-Time Polymorphism?

Runtime par JVM decide karti hai kaunsa method execute hoga.

Method Overriding se hota hai.

Example

```
Animal a = new Dog();
```

```
a.sound();
```

Output

Bark

27. What is Encapsulation?

Data aur methods ko ek class me wrap karna.

Data ko private rakhna.

Access getter/setter se dena.

Example

```
class Student {  
  
    private String name;  
  
    public void setName(String name) {  
        this.name = name;  
    }  
  
    public String getName() {  
        return name;  
    }  
}
```

28. What is Inheritance?

Ek class dusri class ki properties aur methods inherit karti hai.

Keyword

extends

Example

```
class Animal {  
  
    void eat() {  
        System.out.println("Eating");  
    }  
  
}  
  
class Dog extends Animal {  
  
}
```

29. What is Polymorphism?

Ek hi method alag-alag behavior dikhata hai.

Types

- Compile Time
- Run Time

Example

```
Animal a = new Dog();
```

```
a.sound();
```

30. What is Abstraction?

Implementation hide karna aur sirf important details dikhana.

Achieve by

- Abstract Class
- Interface

Example

```
abstract class Vehicle {  
  
    abstract void start();  
  
}
```

31. Difference Between Abstract Class and Interface

Abstract Class	Interface
abstract keyword	interface keyword
Constructor hota hai	Constructor nahi
Variables ho sakte hain	Variables by default public static final

Multiple inheritance
nahi

Multiple inheritance possible

32. Can we create an Object of an Abstract Class?

No.

Abstract class ka object nahi bana sakte.

Example

```
abstract class Animal {  
  
}
```

Wrong

```
Animal a = new Animal();
```

33. What is Interface?

100% abstraction ke liye use hota tha (Java 8 se default/static methods aa gaye).

Keyword

interface

Example

```
interface Animal {  
  
    void sound();  
  
}
```

```
class Dog implements Animal {
```

```
    public void sound() {
```

```
        System.out.println("Bark");  
    }  
}
```

34. Difference Between == and equals()

==	equals()
Reference compare	Content compare

Example

```
String s1 = "Java";  
  
String s2 = "Java";  
  
System.out.println(s1 == s2);  
  
System.out.println(s1.equals(s2));
```

Output

true

true

Example

```
String s1 = new String("Java");  
  
String s2 = new String("Java");
```

Output

==

false

`equals()`

`true`

35. What is String?

String characters ka collection hai.

Java me String immutable hoti hai.

Example

```
String name = "Gourav";
```

36. Why is String Immutable?

Security

Performance

String Pool

Thread Safety

Example

```
String s = "Java";
```

```
s.concat(" Programming");
```

```
System.out.println(s);
```

Output

Java

Original String change nahi hui.

37. Difference Between String, StringBuilder and StringBuffer

String	StringBuilder	StringBuffer
Immutable	Mutable	Mutable
Slow	Fast	Slow
Thread Safe	No	Yes

Example

```
StringBuilder sb = new StringBuilder();  
  
sb.append("Java");
```

38. What is Exception Handling?

Program crash hone se bachane ka mechanism.

Keywords

try

catch

finally

throw

throws

Example

```
try {  
  
    int a = 10 / 0;  
  
} catch (Exception e) {  
  
    System.out.println("Exception");  
}
```

}

39. Difference Between Checked and Unchecked Exception

Checked	Unchecked
Compile Time	Runtime
Handle karna compulsory	Optional

Checked

IOException

Unchecked

ArithmeticException

NullPointerException

ArrayIndexOutOfBoundsException

40. Explain `try`, `catch`, `finally`, `throw`, and `throws`

`try`

Risky code.

`try{`

`}`

`catch`

Exception handle karta hai.

```
catch(Exception e){  
}
```

finally

Hamesha execute hota hai.

```
finally{  
}
```

throw

Khud exception throw karna.

```
throw new ArithmeticException();
```

throws

Method declare karta hai ki exception aa sakti hai.

```
void test() throws IOException{  
}
```

41. What is Garbage Collection (GC)?

Answer

Garbage Collection ek automatic memory management process hai jo JVM ke andar hota hai. Ye un objects ki memory free karta hai jinhe program ab use nahi kar raha.

Benefits

- Memory leak kam hoti hai.
- Memory automatically free hoti hai.

- Developer ko manually memory free nahi karni padti.

Example

```
class Demo {  
    public static void main(String[] args) {  
  
        Demo d1 = new Demo();  
        Demo d2 = new Demo();  
  
        d1 = null;  
        d2 = null;  
  
        System.gc(); // GC ko request  
    }  
}
```

Interview Tip: `System.gc()` sirf request karta hai, GC chalna guaranteed nahi hota.

42. What is Collections Framework?

Answer

Collections Framework Java ka ready-made classes aur interfaces ka group hai jo data ko store aur manage karne ke liye use hota hai.

Main Interfaces

Collection

```
|  
|---- List  
|---- Set  
|---- Queue
```

Map (Separate Interface)

Common Classes

- ArrayList
- LinkedList
- HashSet

- TreeSet
 - HashMap
 - Hashtable
-

43. Difference Between Array and ArrayList

Array	ArrayList
Fixed Size	Dynamic Size
Primitive + Object	Objects Only
Faster	Slightly Slow
Length Property	size() Method

Array

```
int arr[] = {10,20,30};
```

ArrayList

```
ArrayList<Integer> list = new ArrayList<>();
```

```
list.add(10);
```

```
list.add(20);
```

44. Difference Between ArrayList and LinkedList

ArrayList	LinkedList
Dynamic Array	Doubly Linked List
Fast Searching	Fast Insertion/Deletion
Slow Insert/Delete	Fast Insert/Delete

45. Difference Between HashMap and Hashtable

HashMap	Hashtable
Not Thread Safe	Thread Safe
Fast	Slow
One Null Key Allowed	Null Not Allowed

Example

```
HashMap<Integer,String> map = new HashMap<>();  
  
map.put(1,"Java");
```

46. Difference Between HashSet and TreeSet

HashSet	TreeSet
Unordered	Sorted
Faster	Slower
Uses Hash Table	Uses Tree

Example

```
HashSet<Integer> hs = new HashSet<>();  
  
hs.add(50);  
hs.add(20);  
hs.add(10);
```

Output

50 10 20

TreeSet

```
TreeSet<Integer> ts = new TreeSet<>();
```

```
ts.add(50);
```

```
ts.add(20);
```

```
ts.add(10);
```

Output

10 20 50

47. What is Multithreading?

Answer

Ek hi program me multiple threads ka ek saath execute hona Multithreading kehlata hai.

Benefits

- Fast Execution
- Better CPU Utilization
- Multiple Tasks Together

Example

```
class Demo extends Thread{
```

```
    public void run(){
```

```
        System.out.println("Thread Running");
```

```
    }
```

```
}
```

```
public class Main{
```

```
    public static void main(String[] args){
```

```
        Demo d=new Demo();
```

```
d.start();  
  
}  
  
}
```

48. Difference Between Process and Thread

Process	Thread
Independent Program	Part of Process
Heavyweight	Lightweight
Own Memory	Shared Memory

Example

Chrome Browser

Chrome

```
|  
|---- Tab 1  
|---- Tab 2  
|---- Tab 3
```

Chrome = Process

Tabs = Threads

49. What is Synchronization?

Answer

Synchronization ka use multiple threads ko same resource ko safely access karne ke liye hota hai.

Keyword

synchronized

Example

```
class Demo{  
  
    synchronized void print(){  
  
        System.out.println("Printing");  
  
    }  
  
}
```

50. What are Lambda Expressions and Stream API?

Lambda Expression

Anonymous Function.

Java 8 me introduce hua.

Example

```
interface Demo{  
  
    void show();  
  
}  
  
public class Main{  
  
    public static void main(String[] args){  
  
        Demo d=()->System.out.println("Hello");  
  
        d.show();  
  
    }  
  
}
```

```
}
```

Stream API

Collection par operations perform karne ke liye.

Example

```
List<Integer> list = Arrays.asList(10,20,30,40);
```

```
list.stream()
```

```
.filter(x->x>20)
```

```
.forEach(System.out::println);
```

Output

30

40

Bonus Questions

51. What is Wrapper Class?

Primitive datatype ko Object me convert karne wali classes.

Primitive	Wrapper
-----------	---------

int	Integer
-----	---------

char	Character
------	-----------

double	Double
--------	--------

boolean Boolean

Example

Integer a = 100;

52. What is Autoboxing and Unboxing?

Autoboxing

Primitive → Object

int a = 10;

Integer b = a;

Unboxing

Object → Primitive

Integer a = 100;

int b = a;

53. What is Object Class?

Java ki har class indirectly Object class ko inherit karti hai.

Common Methods

toString()

equals()

hashCode()

clone()

wait()

notify()

54. Difference Between HashMap and TreeMap

HashMap	TreeMap
Unordered	Sorted
Faster	Slower
Uses Hash Table	Uses Red-Black Tree

55. Comparable vs Comparator

Comparable

Natural Sorting.

Method

compareTo()

Example

```
Collections.sort(list);
```

Comparator

Custom Sorting.

Method

compare()

Example

```
Collections.sort(list,new MyComparator());
```

56. What is Serialization and Deserialization?

Serialization

Object → File

Deserialization

File → Object

Example

```
class Student implements Serializable{  
  
}
```

57. What is Marker Interface?

Aisi Interface jisme koi method nahi hota.

Examples

Serializable

Cloneable

58. What is Reflection API?

Runtime par class ki information read aur modify karna.

Use

- Spring
- Hibernate
- JUnit

Example

```
Class c = Student.class;
```

59. What is File Handling?

File read/write karna.

Example

```
File file = new File("demo.txt");
```

Read

```
FileReader fr = new FileReader(file);
```

Write

```
FileWriter fw = new FileWriter(file);
```

60. What are Access Modifiers?

Modifier	Same Class	Same Package	Subclasses	Other Package
private	✓	✗	✗	✗
default	✓	✓	✗	✗
protected	✓	✓	✓	✗*
public	✓	✓	✓	✓

protected dusre package me sirf inheritance ke through accessible hota hai.